CO1-AT2

Erorr Analysis Task

Q1) **Analyze Token Recognition and Lexical State Transitions**

(a) Lexeme and Token Analysis

1. Lexemes in order

Given input:

```
int count = 25;
while (count >= 10) {
    count = count - 5;
}
result = count != 0;
```

Lexemes: int, count, =, 25, ;, while, (, count, >=, 10,), {, count, =, count, -, 5, ;, }, result, =, count, !=, 0, ;

2. Token classes

Lexeme	Token class
int, while	Keyword
count, result	Identifier
25, 10, 5, 0	Integer constant
=	Assignment operator
>=, !=	Relational operator
-	Arithmetic operator
;, (,), {, }	Separator

3. Token boundaries

The lexical analyzer scans from left to right, ignores whitespace, and groups characters into valid tokens. Letters form identifiers or keywords, digits form integer constants, and symbols form operators or separators. It uses the longest-match principle to identify the longest valid token.

(b) State-Based Analysis

1. DFA for identifiers and integers

- S0: Start state.
- S1: Identifier state; accepts letters followed by letters or digits.
- S2: Integer state; accepts one or more digits.
- S3: Reads > and checks for =.
- S4: Accepts >=.
- S5: Reads ! and checks for =.
- S6: Accepts !=.

2. Important transitions

Current state	Input	Next state
S0	Letter	S1
S1	Letter/digit	S1
S0	Digit	S2
S2	Digit	S2
S0	>	S3
S3	=	S4
S0	!	S5
S5	=	S6

3. Whitespace, separators and EOF

- Whitespace: Ignored and terminates the current token.
- Separators: Recognized as individual tokens.
- EOF: The scanner completes the pending token and stops.

(c) Longest-Match and Rule Priority

1. Why is >= one token?

The longest-match principle selects the longest valid token. Therefore, >= is recognized as one relational operator instead of separate > and = tokens.

2. Distinguishing != from ! and =

If ! is followed by =, the scanner recognizes != as one relational operator. If = occurs independently, it is an assignment operator. A standalone ! produces a lexical error if unsupported.

3. Rule ordering

Longest-match behavior ensures that multi-character operators are recognized correctly. Keywords are checked against a keyword table after scanning the complete identifier. Rule priority resolves cases where multiple rules match the same length.

(d) Analytical Extension

1. Regular expression for identifiers with underscores

Original:

[A-Za-z][A-Za-z0-9]*

Revised:

[A-Za-z_][A-Za-z0-9_]*

This allows identifiers such as count_value and _result.

2. Adding the keyword do

Add do to the keyword table. The scanner checks the complete lexeme.

- do → Keyword
- double → Identifier

- do2 → Identifier

3. Two test inputs

Test 1:

int total = 50;

while (total >= 20) total = total - 5;

Expected: Correct recognition of keywords, identifiers, integers, operators, and separators.

Test 2:

do_count = 10;

result = do_count != 0;

Expected: do_count is an identifier, 10 and 0 are integer constants, = is an assignment operator, and != is one relational operator.

Conclusion

A lexical analyzer converts a source program into a sequence of tokens by recognizing lexemes, assigning token classes, ignoring whitespace, and applying longest-match rules. A DFA provides a systematic way to recognize identifiers, integer constants, and operators. Correct rule priority ensures that keywords and multi-character operators such as >= and != are recognized accurately.

Q2) **Design a LEX-Based Token Recognizer for a Mini Expression Language**

(a) Token Specification

1. Regular expressions

The regular expressions used in the mini expression language help the lexical analyzer identify different tokens. The keywords are let, print, and if. Identifiers begin with a letter or underscore and may contain letters, digits, and underscores, represented by [A-Za-z_][A-Za-z0-9_]*. Integer constants use [0-9]+, while floating-point constants use [0-9]+\.[0-9]+. Arithmetic operators include +, -, *, /, and %. Comparison operators include <, <=, >, >=, ==, and !=. The assignment operator is =, and separators include parentheses, braces, semicolons, and commas. Whitespace, including spaces, tabs, and newlines, is ignored using [\t\n]+. Single-line comments begin with // and continue to the end of the line, represented by //.*.

2. Overlapping patterns

Keywords overlap with identifiers. For example, print matches both patterns. LEX resolves equal-length matches using rule order, so keyword rules should appear before the identifier rule.

3. Floating-point and integer patterns

The floating-point pattern must be placed before the integer pattern. For example, 12.5 should be recognized as one floating-point constant rather than an integer followed by other characters.

(b) LEX Program Design

```
%{
#include <stdio.h>
%}
%%
"let"|"print"|"if"          { printf("KEYWORD: %s\n", yytext); }
[0-9]+\.[0-9]+              { printf("FLOAT: %s\n", yytext); }
[0-9]+                      { printf("INTEGER: %s\n", yytext); }
[A-Za-z_][A-Za-z0-9_]*      { printf("IDENTIFIER: %s\n", yytext); }
```

```
"<="|">="|"=="|"!="|"<"|">"    { printf("REL_OP: %s\n", yytext); }
[+\-*/%]                        { printf("ARITH_OP: %s\n", yytext); }
"="                             { printf("ASSIGN_OP: %s\n", yytext); }
[(){};,]                 { printf("SEPARATOR: %s\n", yytext); }
"//".*                   { /* Ignore comments */ }
[ \t\n]+                 { /* Ignore whitespace */ }
.                        { printf("INVALID: %s\n", yytext); }
int main()
{
   yylex();
   return 0;
}
int yywrap()
{
   return 1;
}
```

The declaration section includes the header file, the rule section recognizes tokens and prints their types, and the auxiliary section contains main() and yywrap().

(c) Token Stream Validation

1. Expected token sequence

For the given input:

```
let total = 125;
let rate = 12.5;
print(total + rate * 2);
if (total >= 100) {
// display the calculated value
```

```
print(total);
}
```

Expected tokens:

KEYWORD: let

IDENTIFIER: total

ASSIGN_OP: =

INTEGER: 125

SEPARATOR: ;

KEYWORD: let

IDENTIFIER: rate

ASSIGN_OP: =

FLOAT: 12.5

SEPARATOR: ;

KEYWORD: print

SEPARATOR: (

IDENTIFIER: total

ARITH_OP: +

IDENTIFIER: rate

ARITH_OP: *

INTEGER: 2

SEPARATOR:)

SEPARATOR: ;

KEYWORD: if

SEPARATOR: (

IDENTIFIER: total

REL_OP: >=

INTEGER: 100

SEPARATOR:)

SEPARATOR: {

KEYWORD: print

SEPARATOR: (

IDENTIFIER: total

SEPARATOR:)

SEPARATOR: ;

SEPARATOR: }

The comment // display the calculated value and whitespace are ignored.

2. Token categories

- Keywords: let, print, if
- Identifiers: total, rate
- Constants: 125, 12.5, 2, 100
- Operators: =, +, *, >=
- Separators: (,), {, }, ;
- Comment: // display the calculated value (ignored)

3. print versus printer

LEX selects the longest matching lexeme. The keyword rule matches print, but printer matches the longer identifier pattern. Therefore, print is a keyword, while printer is an identifier.

(d) Testing and Evaluation

Test 1: Floating-point value

Input:

let price = 45.75;

Expected tokens:

KEYWORD: let

IDENTIFIER: price

ASSIGN_OP: =

FLOAT: 45.75

SEPARATOR: ;

Test 2: Multi-character relational operator

Input:

if (a != 10)

Expected tokens:

KEYWORD: if

SEPARATOR: (

IDENTIFIER: a

REL_OP: !=

INTEGER: 10

SEPARATOR:)

Test 3: Identifier containing underscores

Input:

let total_sum = 50;

Expected tokens:

KEYWORD: let

IDENTIFIER: total_sum

ASSIGN_OP: =

INTEGER: 50

SEPARATOR: ;

These test cases verify floating-point recognition, multi-character operator handling, and identifiers containing underscores. They help ensure that the LEX specification generates the correct token stream.

Q3) **Analyze the Interaction Between Lexical Analysis and Subsequent Compiler Phases**

(a) Lexical Analysis

1. Lexemes and their classification

Given program:

```
int calculate(int a, int b) {
   int result;
   result = a + b * 10;
   return result;
}
```

The lexemes are classified as follows:

- Keywords: int, return
- Identifiers: calculate, a, b, result
- Constants: 10
- Operators: =, +, *
- Separators: (,), {, }, ,, ;

2. Token stream

KEYWORD(int)

ID(calculate)

SEPARATOR(()

KEYWORD(int)

ID(a)

SEPARATOR(,)

KEYWORD(int)

ID(b)

SEPARATOR())

SEPARATOR({)

KEYWORD(int)

ID(result)

SEPARATOR(;)

ID(result)

ASSIGN_OP(=)

ID(a)

PLUS(+)

ID(b)

MULTIPLY(*)

CONSTANT(10)

SEPARATOR(;)

KEYWORD(return)

ID(result)

SEPARATOR(;)

SEPARATOR(})

3. Treatment of whitespace and punctuation

Whitespace, such as spaces and newlines, is ignored by the lexical analyzer. Punctuation symbols such as parentheses, braces, commas, and semicolons are recognized as separate tokens that help define the program's structure.

(b) Symbol Table and Syntax Analysis

1. Symbol table information

The symbol table stores information about identifiers, such as their names, data types, scope, and roles.

Identifier	Type	Role
calculate	Function	Function name
a	Integer	First parameter
b	Integer	Second parameter
result	Integer	Local variable

2. Syntax analyzer

The syntax analyzer uses the token stream to check whether the program follows the grammar rules.

- Function declaration: Recognizes int calculate(int a, int b).
- Parameter list: Identifies parameters a and b.
- Variable declaration: Recognizes int result;.
- Assignment statement: Parses result = a + b * 10;.
- Return statement: Recognizes return result;.

3. Role of **lexical analysis**

The lexical analyzer converts source code into tokens and supplies them to the syntax analyzer. This allows the syntax analyzer to focus on grammatical structure rather than individual characters.

(c) Intermediate Representation Analysis

1. Three-address code

For the statement result = a + b * 10;, the three-address code is:

t1 = b * 10

t2 = a + t1

result = t2

2. Operator precedence

Multiplication has higher precedence than addition. Therefore, b * 10 must be evaluated before adding a.

3. Temporary variables

Two temporary variables, t1 and t2, are introduced to store intermediate results. They simplify the expression into smaller operations.

(d) Phase Dependency Analysis

1. Information passed between phases

Lexical analysis supplies tokens and their lexemes to syntax analysis. Syntax analysis checks grammatical structure and provides a syntax tree or similar representation to semantic analysis for type checking and other meaning-related checks.

2. Incorrect classification of *

If * is incorrectly classified as an identifier, the syntax analyzer may reject the expression because an identifier cannot replace the multiplication operator in that position.

3. Error propagation

An incorrect token can cause syntax errors, incorrect syntax trees, and failures in semantic analysis or code generation. Therefore, accurate lexical analysis is essential for the later compiler phases.

(e) Advanced Compiler Perspective

1. Improvements to lexical analysis

1. Use well-defined regular expressions and longest-match rules to recognize tokens correctly.
2. Add token validation and clear error reporting for invalid characters and malformed constants.

2. Importance of line and column numbers

Line and column numbers help the compiler identify the exact location of an error. This makes debugging easier and allows programmers to correct errors quickly.

3. Test for multi-character operators

Test input:

if (a >= 10 && b != 5)

Expected recognition includes >= as one relational operator and != as one relational operator. If logical operators are supported, && should also be recognized as one token.

Conclusion

Lexical analysis converts source code into tokens, syntax analysis checks grammatical structure, semantic analysis verifies meaning, and intermediate code generation produces a simplified representation. Errors in lexical analysis can affect every subsequent compiler phase.